Reporte: 2p ejercicio 17 (pe)

En este programa se implementa el algoritmo de ordenamiento Merge Sort (Ordenamiento por Mezcla) bajo el paradigma de Programación Estructurada (PE). El programa permite ordenar vectores de enteros, caracteres y estructuras personalizadas (Estudiante) a través de un menú interactivo en consola.

Código Completo:

```
#include <iostream>
#include <vector>
#include <string>
#include <limits>

struct Estudiante {
    int id;
    std::string nombre;
    float promedio;
};

void mergeEnteros(std::vector<int>& arr, int izq, int medio, int der) {
    int n1 = medio - izq + 1;
    int n2 = der - medio;
    std::vector<int> L(n1), R(n2);
    for (int i = 0; i < n1; i++) L[i] = arr[izq + i];
    for (int j = 0; j < n2; j++) R[j] = arr[medio + 1 + j];
    int i = 0, j = 0, k = izq;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) { arr[k] = L[i]; i++; }
        else { arr[k] = R[j]; j++; }
        k++;
    }
    while (i < n1) { arr[k] = L[i]; i++; k++; }
    while (j < n2) { arr[k] = R[j]; j++; k++; }
}

void mergeSortEnteros(std::vector<int>& arr, int izq, int der) {
    if (izq < der) {
        int medio = izq + (der - izq) / 2;
        mergeSortEnteros(arr, izq, medio);
        mergeSortEnteros(arr, medio + 1, der);
        mergeEnteros(arr, izq, medio, der);
    }
}

void mergeCaracteres(std::vector<char>& arr, int izq, int medio, int der) {
    int n1 = medio - izq + 1;
    int n2 = der - medio;
    std::vector<char> L(n1), R(n2);
    for (int i = 0; i < n1; i++) L[i] = arr[izq + i];
    for (int j = 0; j < n2; j++) R[j] = arr[medio + 1 + j];
    int i = 0, j = 0, k = izq;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) { arr[k] = L[i]; i++; }
        else { arr[k] = R[j]; j++; }
```

```
        k++;
    }
    while (i < n1) { arr[k] = L[i]; i++; k++; }
    while (j < n2) { arr[k] = R[j]; j++; k++; }
}

void mergeSortCaracteres(std::vector<char>& arr, int izq, int der) {
    if (izq < der) {
        int medio = izq + (der - izq) / 2;
        mergeSortCaracteres(arr, izq, medio);
        mergeSortCaracteres(arr, medio + 1, der);
        mergeCaracteres(arr, izq, medio, der);
    }
}

void mergeEstudiantes(std::vector<Estudiante>& arr, int izq, int medio, int der) {
    int n1 = medio - izq + 1;
    int n2 = der - medio;
    std::vector<Estudiante> L(n1), R(n2);
    for (int i = 0; i < n1; i++) L[i] = arr[izq + i];
    for (int j = 0; j < n2; j++) R[j] = arr[medio + 1 + j];
    int i = 0, j = 0, k = izq;
    while (i < n1 && j < n2) {
        if (L[i].id <= R[j].id) { arr[k] = L[i]; i++; }
        else { arr[k] = R[j]; j++; }
        k++;
    }
    while (i < n1) { arr[k] = L[i]; i++; k++; }
    while (j < n2) { arr[k] = R[j]; j++; k++; }
}

void mergeSortEstudiantes(std::vector<Estudiante>& arr, int izq, int der) {
    if (izq < der) {
        int medio = izq + (der - izq) / 2;
        mergeSortEstudiantes(arr, izq, medio);
        mergeSortEstudiantes(arr, medio + 1, der);
        mergeEstudiantes(arr, izq, medio, der);
    }
}

int main() {
    int opcion = 0;
    do {
        std::cout << "\n======================================\n";
        std::cout << "   MENÚ INTERACTIVO - MERGE SORT (PE)   \n";
        std::cout << "======================================\n";
        std::cout << "1. Ordenar conjunto de Enteros\n";
        std::cout << "2. Ordenar conjunto de Caracteres\n";
        std::cout << "3. Ordenar nuevo tipo de dato (Estudiantes por ID)\n";
        std::cout << "4. Salir\n";
        std::cout << "Seleccione una opción: ";
        if (!(std::cin >> opcion)) break;

        if (opcion == 1) {
            int n;
            std::cout << "Cantidad de números enteros a ingresar: ";
            std::cin >> n;
            if (n <= 0) continue;
```

```
            std::vector<int> arr(n);
            std::cout << "Ingrese los " << n << " números:\n";
            for (int i = 0; i < n; i++) {
                std::cout << "Elemento [" << i + 1 << "]: ";
                std::cin >> arr[i];
            }
            mergeSortEnteros(arr, 0, n - 1);
            std::cout << "\n¡Vector de enteros ordenado exitosamente!\n[ ";
            for (int i = 0; i < n; i++) std::cout << arr[i] << " ";
            std::cout << "]\n";
        }
        else if (opcion == 2) {
            int n;
            std::cout << "Cantidad de caracteres a ingresar: ";
            std::cin >> n;
            if (n <= 0) continue;
            std::vector<char> arr(n);
            std::cout << "Ingrese los " << n << " caracteres:\n";
            for (int i = 0; i < n; i++) {
                std::cout << "Caracter [" << i + 1 << "]: ";
                std::cin >> arr[i];
            }
            mergeSortCaracteres(arr, 0, n - 1);
            std::cout << "\n¡Vector de caracteres ordenado exitosamente!\n[ ";
            for (int i = 0; i < n; i++) std::cout << arr[i] << " ";
            std::cout << "]\n";
        }
        else if (opcion == 3) {
            int n;
            std::cout << "Cantidad de estudiantes a registrar: ";
            std::cin >> n;
            if (n <= 0) continue;
            std::vector<Estudiante> estudiantes(n);
            for (int i = 0; i < n; i++) {
                std::cout << "\nEstudiante " << i + 1 << ":\n";
                std::cout << "  ID (número entero para ordenar): ";
                std::cin >> estudiantes[i].id;
                std::cout << "  Promedio: ";
                std::cin >> estudiantes[i].promedio;
                std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
                std::cout << "  Nombre: ";
                std::getline(std::cin, estudiantes[i].nombre);
            }
            mergeSortEstudiantes(estudiantes, 0, n - 1);
            std::cout << "\n--- ESTUDIANTES ORDENADOS POR ID (ASCENDENTE) ---\n";
            for (int i = 0; i < n; i++) {
                std::cout << "ID: " << estudiantes[i].id << " | Nombre: " <<
estudiantes[i].nombre << " | Promedio: " << estudiantes[i].promedio << "\n";
            }
        }
        else if (opcion == 4) {
            std::cout << "Saliendo del programa... ¡Hasta luego!\n";
        }
        else {
            std::cout << "Opción no válida. Intente de nuevo.\n";
        }

    } while (opcion != 4);
```

```
    return 0;
}
```

Explicación del Código:

1. Estructura de Datos Personalizada (struct Estudiante)
Define un tipo de dato compuesto con los campos id, nombre y promedio. Permite almacenar información heterogénea de un alumno para ser ordenada posteriormente por su ID.

2. Estrategia 'Divide y Vencerás' (Funciones mergeSort...)
Las funciones mergeSortEnteros, mergeSortCaracteres y mergeSortEstudiantes dividen recursivamente el vector a la mitad (calculando medio = izq + (der - izq) / 2) hasta obtener subvectores de tamaño 1 o 0.

3. Proceso de Mezcla y Combinación (Funciones merge...)
Crea subvectores temporales L (izquierda) y R (derecha), copia los elementos y los compara uno a uno para reescribir el vector original en orden ascendente. Cuenta con funciones dedicadas para cada tipo de dato (int, char y Estudiante).

4. Menú Interactivo en la Función Principal (main)
Implementa un ciclo do-while que permite al usuario seleccionar qué tipo de datos desea ingresar y ordenar. Incluye la limpieza del búfer de entrada (cin.ignore) al leer cadenas con getline() para evitar capturar saltos de línea no deseados.